ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO

Trabalho Prático de

Segurança Informática

2024/2025

Trabalho Elaborado por:

Diogo Fernando Águia Costa (8240696@estg.ipp.pt)

Curso de:

LSIRC - Licenciatura em Segurança informática em redes de computadores.

Docentes:

Silvestre Malta (slm@estg.ipp.pt)

João Oliveira (jpo@estg.ipp.pt)

Felgueiras, 31 de maio de 2025

Conteúdo

1 Introdução **7**

1.1 Funcionamento e Justificação Técnica 7

2 Teoria da Criptografia **9**

2.1 Tipos de Criptografia 9

2.2 Algoritmos de Criptografia 10

2.2.1 Criptografia Simétrica 10

2.2.2 Criptografia Assimétrica 10

2.2.3 Criptografia de Hash 10

2.2.4 Criptografia de Fluxo 10

2.2.5 Criptografia de Curvas Elípticas 11

2.3 Modos de Operação 11

2.3.1 Modos no AES 11

2.3.2 Modos no DES 11

2.3.3 Modos no 3DES 11

2.3.4 Modos no Blowfish 11

2.3.5 Nota sobre o RC4 11

2.4 Considerações Finais 12

3 Programa AES-256-GCM **13**

3.1 Introdução ao Capítulo 13

3.2 Justificação da Escolha 13

3.3 Abordagem ao Problema 13

3.4 Menu Principal 14

3.5 Ficheiros 14
3.6 Mensagens 16
3.7 Ajuda 17
3.8 Código do Programa 19
3.8.1 Estrutura do Código 19
3.8.2 Bibliotecas Utilizadas 19

4 Conclusão **21**

5 Bibliografia **23**

Lista de Figuras

3.1 Menu Principal 14

3.2 Seleção da Opção "Ficheiros" no Menu Principal 14

3.3 Seleção de Ficheiro 15

3.4 Zona de "Drop" para Ficheiros 15

3.5 Funcionalidades na Secção de Ficheiros 15

3.6 Exemplo de Ficheiro Encriptado 16

3.7 Painel de Entrada de Mensagem 16

3.8 Painel de Saída da Mensagem 16

3.9 Exemplo de Mensagem Encriptada 17

3.10 Exemplo de Mensagem Desencriptada 17

3.11 Menu da Aba Ajuda 18

3.12 Janela Informativa - Sobre 18

3.13 Conteúdo da Aba Sobre 18

3.14 Estrutura dos *packages* do código em Java 19

Introdução

O presente trabalho foi desenvolvido no âmbito da unidade curricular de *Segurança Informática*, pertencente ao plano de estudos da *Licenciatura em Segurança Informática em Redes de Computadores (LSIRC)*, lecionada no segundo semestre do primeiro ano do curso. O principal objetivo consistiu no aprofundamento dos conhecimentos adquiridos na unidade curricular, bem como no desenvolvimento de competências na área da programação.

Inicialmente, estava prevista a utilização da linguagem de programação *Python*, considerada mais apropriada para este tipo de projeto, dada a sua vasta gama de bibliotecas dedicadas à criptografia. Contudo, devido à sobreposição com o desenvolvimento de outro projeto, pertencente à unidade curricular de *Paradigmas da Programação*, optou-se por utilizar a linguagem *Java*. Embora esta apresente uma abordagem menos direta à criptografia, disponibiliza igualmente recursos robustos para a implementação de sistemas de segurança.

O projeto desenvolvido permite a introdução de qualquer tipo de documento ou mensagem, os quais são encriptados e desencriptados através do algoritmo de criptografia **AES-256**, utilizando o modo **Galois/Counter Mode (GCM)**. Esta abordagem foi adotada com base na sua elevada segurança e eficiência, sendo amplamente reconhecida como uma das soluções mais fiáveis na atualidade no âmbito da criptografia simétrica.

1.1 Funcionamento e Justificação Técnica

Durante a execução do programa, é solicitada ao utilizador a introdução de uma chave secreta, a qual será utilizada nos processos de encriptação e desencriptação dos dados. A utilização da mesma chave em ambos os processos é obrigatória, uma vez que qualquer divergência impedirá a correta recuperação da informação original. Nestes casos, o programa rejeita o conteúdo e bloqueia o processo de desencriptação. Além disso, todos os erros que possam ocorrer durante os processos de encriptação e desencriptação são tratados de forma adequada e informativa.

A escolha do algoritmo **AES-256-GCM** deve-se à sua reconhecida robustez, combinando segurança e desempenho. Este é atualmente utilizado em diversos contextos práticos, como em sistemas de criptografia de dados, bases de dados, redes VPN, serviços de correio eletrónico, plataformas de armazenamento na nuvem, entre outros. O modo GCM, em particular, garante não apenas a confidencialidade dos dados, mas também a sua integridade, o que o torna especialmente eficaz em ambientes que exigem elevados níveis de segurança.

Ainda assim, importa salientar que todos os algoritmos de criptografia apresentam vantagens e limitações, sendo a sua aplicação dependente das necessidades específicas de cada sistema e do contexto em que são utilizados.

Teoria da Criptografia

A criptografia surgiu como uma ferramenta essencial para proteger informações e garantir a segurança das comunicações e dos dados, independentemente do meio em que estes se encontram. A sua utilização visa assegurar a confidencialidade, a integridade, a autenticidade e a resistência contra diversos tipos de ataques.

2.1 Tipos de Criptografia

A criptografia pode ser dividida em diferentes categorias, consoante os princípios utilizados e os objetivos pretendidos. De forma análoga à escolha de um continente num mundo imaginário, cada tipo de criptografia representa uma abordagem distinta, com características e aplicações próprias. Os principais tipos são:

- **Criptografia Simétrica:** Utiliza a mesma chave para encriptar e desencriptar os dados.
- **Criptografia Assimétrica:** Utiliza um par de chaves (pública e privada); a chave pública encripta e a chave privada desencripta.
- **Criptografia de Hash:** Transforma dados de entrada num valor de comprimento fixo, sendo utilizada sobretudo para verificação da integridade dos dados.
- **Criptografia Híbrida:** Combina criptografia simétrica e assimétrica, aproveitando as vantagens de ambas. Normalmente, a parte assimétrica serve para transmitir com segurança a chave simétrica.
- **Criptografia de Fluxo:** Encripta os dados bit a bit ou byte a byte, sendo adequada para transmissões contínuas.
- **Criptografia Homomórfica:** Permite a realização de operações sobre dados encriptados sem necessidade de desencriptação.
- **Criptografia de Curvas Elípticas:** Baseia-se em propriedades matemáticas de curvas elípticas, permitindo criar chaves seguras e eficientes com tamanhos reduzidos.

Cada uma destas categorias apresenta métodos e propósitos distintos, o que justifica a sua utilização consoante os requisitos e características de cada sistema.

2.2 Algoritmos de Criptografia

Tal como cada continente possui vários países, cada tipo de criptografia apresenta diversos algoritmos específicos, adequados a diferentes cenários. Abaixo apresentam-se os principais algoritmos, organizados por categoria.

2.2.1 Criptografia Simétrica

- **AES (Advanced Encryption Standard):** Um dos padrões mais utilizados atualmente, disponível com chaves de 128, 192 ou 256 bits.
- **DES (Data Encryption Standard):** Um algoritmo mais antigo, com chave de 56 bits, atualmente considerado inseguro.
- **3DES (Triple DES):** Versão melhorada do DES, aplica o algoritmo três vezes com chaves diferentes, aumentando a segurança.
- **Blowfish:** Algoritmo de bloco, conhecido pela sua velocidade e flexibilidade.
- **RC4:** Cifrador de fluxo, anteriormente utilizado em diversos protocolos, atualmente considerado obsoleto para muitas aplicações.

2.2.2 Criptografia Assimétrica

- **RSA (Rivest-Shamir-Adleman):** Utilizado amplamente para transmissão segura de dados e assinaturas digitais.
- **ECC (Elliptic Curve Cryptography):** Oferece segurança equivalente à do RSA, mas com chaves de tamanho reduzido, aumentando a eficiência.
- **DSA (Digital Signature Algorithm):** Usado principalmente para assinaturas digitais, garantindo autenticação e integridade.

2.2.3 Criptografia de Hash

- **SHA (Secure Hash Algorithm):** Inclui variantes como SHA-1 (inseguro), SHA-256 e SHA-3.
- **MD5 (Message Digest Algorithm 5):** Muito utilizado no passado, mas atualmente considerado inseguro devido a vulnerabilidades conhecidas.

2.2.4 Criptografia de Fluxo

- **RC4:** Cifrador de fluxo já obsoleto para aplicações modernas.
- **Salsa20 e ChaCha20:** Cifradores de fluxo modernos, reconhecidos pela sua segurança e desempenho.

2.2.5 Criptografia de Curvas Elípticas

- **ECC (Elliptic Curve Cryptography):** Também enquadrada como criptografia assimétrica, baseia-se em curvas elípticas para oferecer maior segurança com menor custo computacional.

A existência de múltiplos algoritmos permite uma grande flexibilidade na escolha da solução criptográfica mais adequada para cada aplicação. Alguns algoritmos destacam-se pela sua robustez, outros pela eficiência, e muitos já foram descontinuados por apresentarem vulnerabilidades.

2.3 Modos de Operação

Dentro de cada algoritmo, especialmente no caso da criptografia simétrica por blocos, existem ainda os chamados *modos de operação*. Estes definem a forma como os blocos de dados são encriptados, funcionando como "cidades dentro dos países", numa analogia à hierarquia de conceitos criptográficos.

Nem todos os algoritmos possuem modos de operação, mas os mais relevantes incluem:

2.3.1 Modos no AES

- **ECB (Electronic Codebook)**
- **CBC (Cipher Block Chaining)**
- **CTR (Counter)**

2.3.2 Modos no DES

- **ECB (Electronic Codebook)**
- **CBC (Cipher Block Chaining)**

2.3.3 Modos no 3DES

- **ECB (Electronic Codebook)**
- **CBC (Cipher Block Chaining)**

2.3.4 Modos no Blowfish

- **ECB (Electronic Codebook)**
- **CBC (Cipher Block Chaining)**

2.3.5 Nota sobre o RC4

O algoritmo RC4, por ser um cifrador de fluxo, não utiliza modos de operação como os algoritmos de bloco.

2.4 Considerações Finais

A escolha do algoritmo e do modo de operação mais adequado depende dos requisitos específicos de segurança, do desempenho desejado e do contexto da aplicação. Mais importante ainda, a utilização correta das práticas criptográficas é fundamental para garantir a proteção eficaz dos dados. Um algoritmo robusto mal implementado pode comprometer todo o sistema de segurança.

Programa AES-256-GCM

3.1 Introdução ao Capítulo

Após a apresentação teórica sobre os principais tipos, algoritmos e modos de operação da criptografia, este capítulo introduz o trabalho prático desenvolvido. O foco principal recai sobre a utilização do algoritmo **AES-256-GCM**, que foi selecionado como base para a implementação do sistema de encriptação e desencriptação.

3.2 Justificação da Escolha

A escolha do algoritmo AES-256-GCM resultou de uma análise comparativa entre diferentes alternativas criptográficas. Considerando os requisitos do projeto nomeadamente a necessidade de encriptar vários tipos de dados concluiu-se que o *Advanced Encryption Standard* (AES) com chave de 256 bits e modo de operação GCM (Galois/Counter Mode) seria a opção mais adequada.

O modo GCM oferece, simultaneamente, confidencialidade e integridade dos dados, através da autenticação da mensagem. Além disso, proporciona um bom desempenho e é amplamente utilizado em aplicações modernas, como serviços de nuvem, VPNs, bases de dados e sistemas de comunicação seguros.

A utilização de uma chave com 256 bits teve como objetivo maximizar a segurança do sistema, em conformidade com os padrões atuais da criptografia avançada.

3.3 Abordagem ao Problema

Durante o planeamento do projeto, considerou-se a necessidade de encriptar diferentes tipos de ficheiros, tais como imagens, documentos de texto, ficheiros de áudio, entre outros. Inicialmente, foi realizada a implementação da encriptação e desencriptação de documentos, assegurando o correto funcionamento do processo com diferentes extensões de ficheiro.

Após concluída essa etapa, foi introduzida a funcionalidade de encriptação e desencriptação de mensagens de texto, de forma a ampliar o leque de dados suportados pelo sistema. A estrutura do programa foi concebida de forma modular, permitindo uma possível expansão futura para outros formatos ou funcionalidades criptográficas adicionais.

3.4 Menu Principal

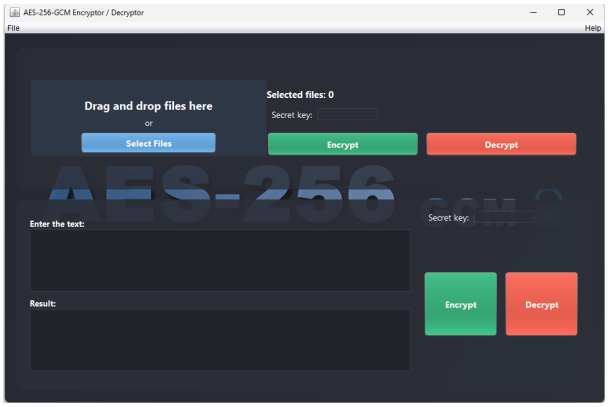

Figura 3.1: Menu Principal

No menu principal, o utilizador pode optar entre duas funcionalidades: encriptação e desencriptação de documentos ou mensagens. Esta interface permite uma navegação simples e direta, facilitando a escolha do tipo de dados a tratar.

3.5 Ficheiros

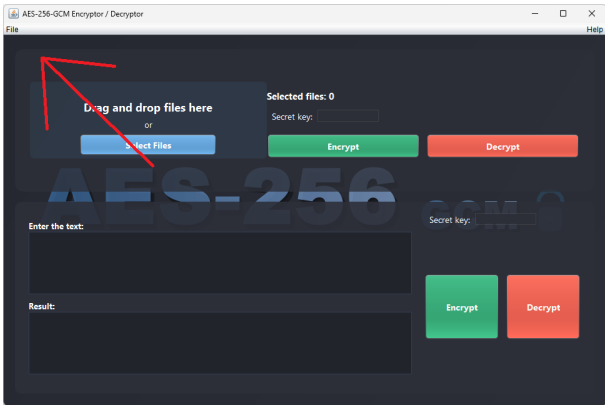

Figura 3.2: Seleção da Opção "Ficheiros" no Menu Principal

Ao selecionar a opção **Ficheiros** (File), é apresentada uma nova interface onde se pode escolher o ficheiro a ser encriptado ou desencriptado. A seleção é feita através de um motor de busca integrado na aplicação,

acessível na barra de menus. Se selecionarmos a saída (Exit) o programa fecha automaticamente.

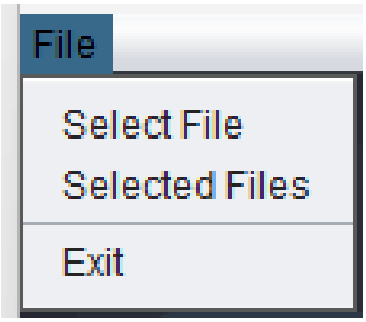

Figura 3.3: Seleção de Ficheiro

Alternativamente, é possível arrastar o ficheiro diretamente para a zona de *drop* disponível na interface, tornando o processo mais rápido e intuitivo.

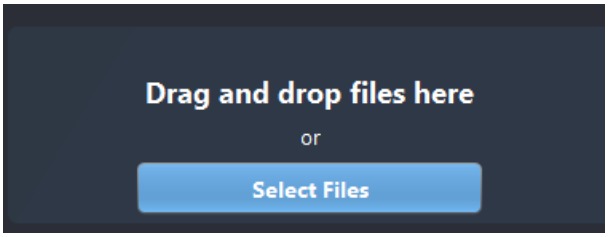

Figura 3.4: Zona de "Drop"para Ficheiros

Após a seleção ou carregamento do documento, o utilizador deve introduzir uma chave secreta, que será utilizada para o processo de encriptação ou desencriptação. Se o ficheiro não estiver encriptado, o programa procederá à encriptação; caso contrário, será feita a desencriptação.

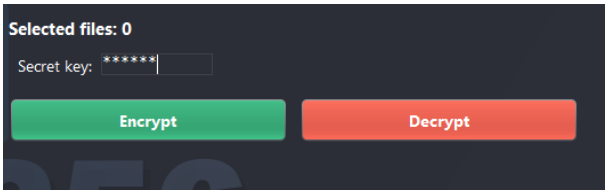

Figura 3.5: Funcionalidades na Secção de Ficheiros

O ficheiro encriptado é gerado num formato específico, com uma extensão ou designação que identifica claramente tratar-se de um ficheiro protegido. Para que a desencriptação seja bem-sucedida, é imprescindível utilizar a mesma chave secreta utilizada no processo de encriptação.

Figura 3.6: Exemplo de Ficheiro Encriptado

Caso ocorra algum erro durante o processo de encriptação ou desencriptação como, por exemplo, a introdução de uma chave incorreta ou ficheiro incompatível, o programa apresenta uma janela de erro e exibe a exceção associada, permitindo ao utilizador identificar a causa do problema de forma clara.

3.6 Mensagens

Na secção dedicada às mensagens, é apresentado um painel de texto onde o utilizador pode introduzir a mensagem a ser encriptada ou desencriptada.

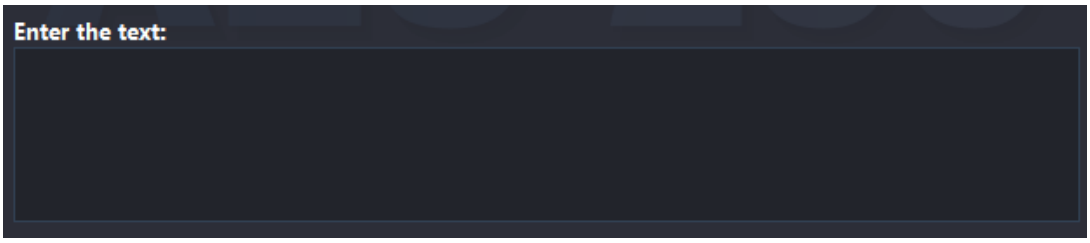

Figura 3.7: Painel de Entrada de Mensagem

Após a introdução do texto, o resultado da operação de encriptação ou desencriptação é apresentado num segundo painel de texto, destinado à saída do processo.

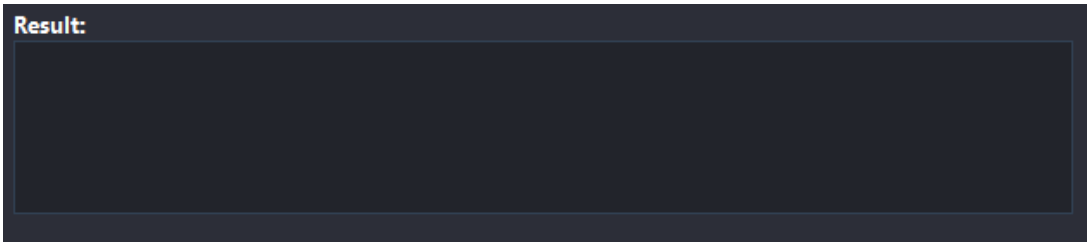

Figura 3.8: Painel de Saída da Mensagem

A aplicação processa a chave introduzida pelo utilizador e executa a operação de encriptação ou desencriptação da mensagem, independentemente do seu tamanho. A Figura 3.9 ilustra um exemplo de mensagem encriptada.

Figura 3.9: Exemplo de Mensagem Encriptada

Para proceder à desencriptação, o conteúdo da área de entrada deve ser substituído pelo texto encriptado. O programa processa a informação e apresenta o conteúdo original, conforme ilustrado na Figura 3.10.

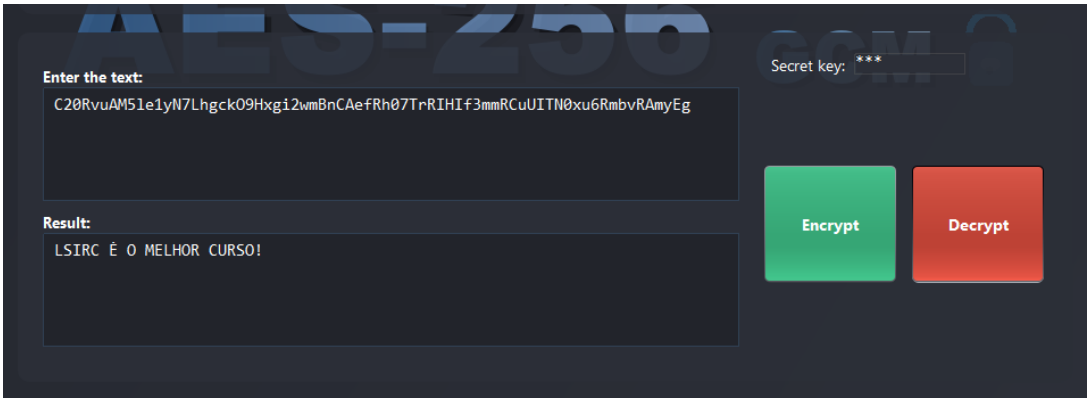

Figura 3.10: Exemplo de Mensagem Desencriptada

Caso ocorra algum erro durante o processo, como uma chave incorreta ou formato inválido, a aplicação apresenta uma janela de erro contendo a exceção associada, permitindo ao utilizador perceber a origem do problema.

3.7 Ajuda

Na aba **Ajuda** (*Help*), a aplicação dispõe de uma opção intitulada *Sobre*, onde é possível consultar informações adicionais sobre o programa.

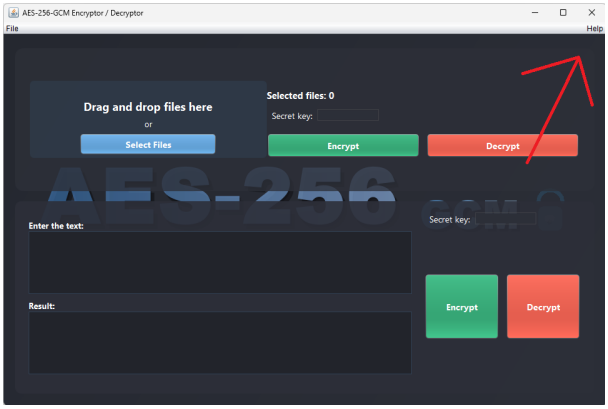

Figura 3.11: Menu da Aba Ajuda

Ao selecionar a opção *Sobre*, é aberta uma janela informativa com detalhes sobre o funcionamento da aplicação, justificações para a escolha do algoritmo AES-256-GCM e referências às fontes de conhecimento utilizadas no desenvolvimento do projeto.

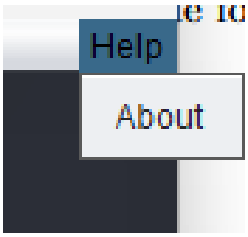

Figura 3.12: Janela Informativa - Sobre

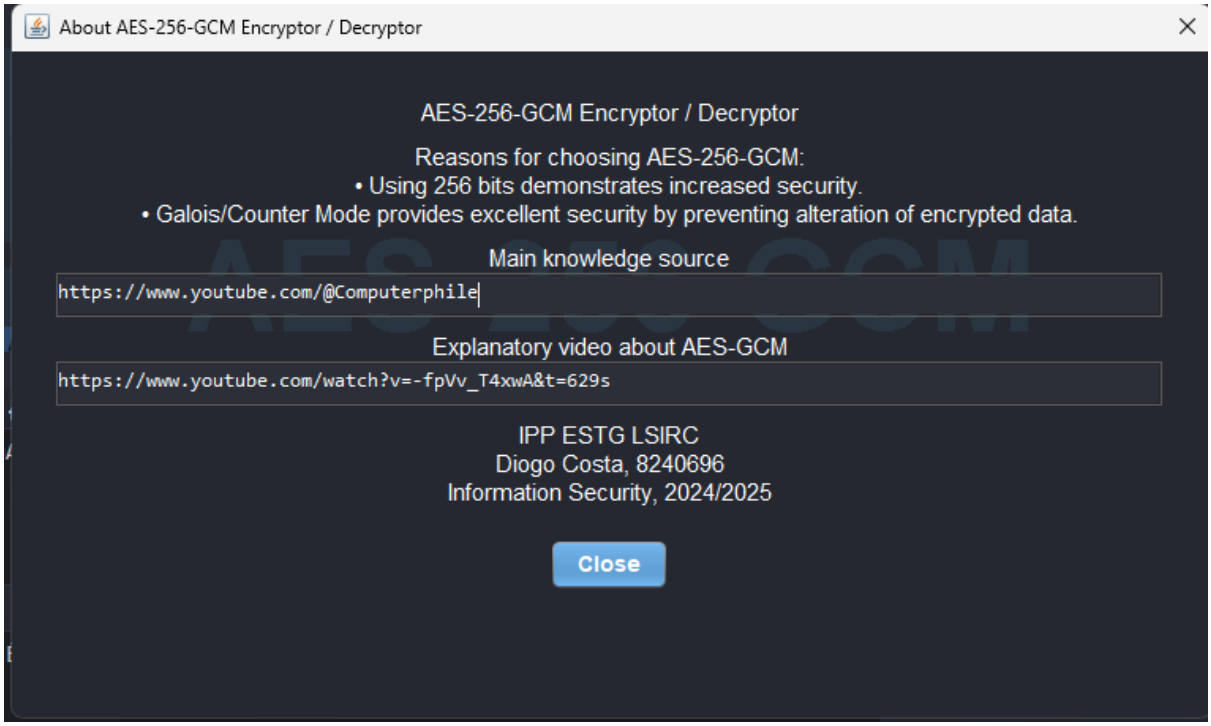

Figura 3.13: Conteúdo da Aba Sobre

3.8 Código do Programa

3.8.1 Estrutura do Código

O código-fonte da aplicação foi organizado em três *packages* principais, de forma a garantir uma separação lógica das funcionalidades:

- **crypto** – Contém as classes responsáveis pelos processos de encriptação e derivação de chaves, nomeadamente `AESCipher` e `KeyDerivation`.
- **ui** – Responsável pela interface gráfica da aplicação. Inclui as classes `AboutDialog` (janela informativa sobre o programa) e `MainWindow` (janela principal da aplicação).
- **util** – Contém funcionalidades auxiliares utilizadas pela interface, como a classe `GuiUtils`, que fornece métodos de apoio à interface gráfica.

A Figura 3.14 apresenta a organização visual destes *packages* no projeto Java.

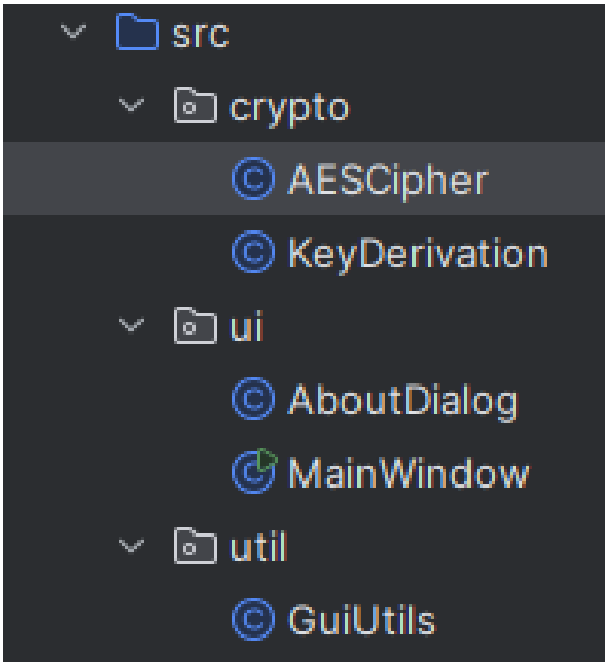

Figura 3.14: Estrutura dos *packages* do código em Java

3.8.2 Bibliotecas Utilizadas

Durante o desenvolvimento da aplicação, foram utilizadas diversas bibliotecas da linguagem de programação Java, essenciais para a construção da interface gráfica, manipulação de ficheiros e implementação da criptografia AES-256-GCM. Estas bibliotecas podem ser organizadas nas seguintes categorias:

- **Interface Gráfica (GUI):**
  - `javax.swing.*` – Conjunto de componentes gráficos para construção da interface do utilizador.

  - `java.awt.*` – Bibliotecas gráficas complementares ao Swing, utilizadas para layout, eventos e elementos visuais.
  - `javax.swing.text.JTextComponent` – Componente base para edição de texto.
- **Manipulação de Ficheiros e Dados:**
  - `java.io.File` – Manipulação de ficheiros no sistema de ficheiros.
  - `java.nio.file.Files` – Operações como leitura e escrita de ficheiros.
  - `java.nio.file.Paths` – Manipulação de caminhos de ficheiros.
  - `java.util.List` e `java.util.ArrayList` – Estruturas de dados utilizadas para armazenar listas dinâmicas.
  - `java.awt.datatransfer.DataFlavor` e `java.awt.dnd.*` – Funcionalidades de arrastar e largar (drag and drop).
- **Criptografia:**
  - `javax.crypto.Cipher` – Classe principal para operações de encriptação e decriptação.
  - `javax.crypto.spec.GCMParameterSpec` – Especificação dos parâmetros para o modo GCM (vetor de inicialização e tamanho do tag de autenticação).
  - `javax.crypto.spec.SecretKeySpec` – Definição de chaves criptográficas baseadas em arrays de bytes.
  - `java.security.SecureRandom` – Geração de vetores de inicialização aleatórios de forma segura.
  - `java.security.MessageDigest` – Operações de hash, como SHA-256, na derivação de chaves.
  - `java.security.NoSuchAlgorithmException` – Tratamento de exceções quando um algoritmo de hash não é suportado.
  - `java.util.Base64` – Conversão de dados binários para representação textual.
  - `java.util.Arrays` – Manipulação de arrays.
- **Pacotes Personalizados:**
  - `crypto.AESCipher` – Classe responsável pela encriptação e decriptação com AES-256-GCM.
  - `crypto.KeyDerivation` – Classe para derivação de chaves seguras.
  - `ui.AboutDialog` e `ui.MainWindow` – Componentes da interface do utilizador.
  - `util.GuiUtils` – Métodos utilitários para interface gráfica.

Estas bibliotecas permitiram desenvolver uma aplicação funcional, segura e com uma interface intuitiva para o utilizador.

Conclusão

Manter as nossas informações seguras é de extrema importância, especialmente na era digital em que vivemos, onde a proteção da privacidade e dos dados pessoais é uma prioridade constante. Para garantir essa segurança, podemos recorrer a técnicas de criptografia robustas como o AES-GCM, que asseguram que apenas as pessoas autorizadas possam aceder aos dados sensíveis.

No entanto, a segurança de um sistema de criptografia não depende apenas do algoritmo utilizado. O modo como as chaves são geridas é igualmente crucial a sua força torna-se irrelevante se forem facilmente acessíveis por utilizadores não autorizados. A correta implementação dos protocolos de segurança, a prevenção contra ataques de força bruta e a mitigação de possíveis fugas de informação são fatores determinantes para a eficácia do sistema. Para além da criptografia, uma estratégia de segurança abrangente deve incluir mecanismos de controlo de acesso, autenticação de utilizadores e monitorização de comportamentos suspeitos. Quando combinamos estes elementos com a criptografia, conseguimos assegurar que os dados permanecem confidenciais e disponíveis sempre que necessário.

Relativamente ao trabalho que desenvolvi, reconheço que existem aspetos que poderiam ser melhorados. A interface gráfica (GUI), por exemplo, poderia estar mais bem organizada e orientada para a experiência do utilizador. A usabilidade do programa pode também ser melhorada, tornando-o mais intuitivo e acessível.

No que diz respeito à segurança, uma das melhorias que considerei seria a introdução de um sistema de autenticação com login de utilizador. Cada conta registada poderia ter uma chave secreta única associada, utilizada automaticamente no processo de encriptação e decriptação durante a sessão, eliminando assim a vulnerabilidade associada à inserção manual da chave.

Outra melhoria possível seria a eliminação automática do ficheiro original após a encriptação ou decriptação, de forma a evitar que o documento não encriptado permaneça no mesmo diretório. A manutenção simultânea do ficheiro original e do encriptado compromete a segurança e vai contra a simplicidade e eficácia que a aplicação pretende oferecer.

Em suma, este projeto proporcionou uma oportunidade prática para explorar os conceitos de criptografia

moderna e refletir sobre a importância de integrar segurança, usabilidade e boas práticas de desenvolvimento numa solução eficaz.

Bibliografia

- https://chatgpt.com/
- https://stackoverflow.com/
- https://www.youtube.com/@Computerphile
- https://github.com/
- https://www.javaprogressivo.net/
- https://crypto.stackexchange.com/
- https://www.guj.com.br/
- https://www.devmedia.com.br/
- https://coderanch.com/
- https://www.javatpoint.com/
- https://www.formdev.com/
- https://softkube.com/